RADAR: A Resilient Anomaly Detection and Attack Radius Inference Framework for RPL-Based IoT Networks

SDP PROJECT PROPOSAL

Submitted in partial fulfillment of the requirement for the award of the Degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCES

by

22BCE8802 - SANJEEVANI PANDIT
22BCE7231 - SONAKSHI
22BCE7561 - G KEERTHI CHOWDARY

Under the Guidance of
DR. SHALINI SUBHRAMANI

VIT-AP UNIVERSITY , AMARAVATI- 522237

S. No.	Contents	Page No.
1	Introduction	4
2	Problem Statement	5
3	Objectives of the Project	7
4	Scope of the Project	8
5	Proposed Methodology	8
6	Tools and Technologies	19
7	Performance Evaluation	20
8	Project Timeline (January–May)	21
9	Expected Outcomes	22
10	Conclusion	23

LIST OF TABLES

Table No.	Title of Table	Page No.
Table 1	Classification of RPL Routing Attacks	5,6
Table 2	Comparison of Traditional and AI-Based Defense Mechanisms	6,7
Table 3	Intent Inference	14
Table 4 Table 5 Table 6	Comparison between existing and proposed methodologies Performance Metrics Used for Evaluation Project Timeline (January–May)	18,19 20,21 21,22

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Fig. 1	Workflow of proposed methodology	12
Fig. 2	Workflow Outcome of proposed methodology	23

Project Proposal

RADAR: A Resilient Anomaly Detection and Attack Radius Inference Framework for RPL-Based IoT Networks

1. Introduction

The Internet of Things (IoT) has emerged as a foundational technology enabling large-scale sensing, monitoring, and automation across diverse application domains such as smart cities, healthcare systems, industrial automation, and intelligent transportation networks. These systems are composed of low-power, resource-constrained devices that operate over unreliable wireless links and are often deployed in unattended or hostile environments. Ensuring reliable, energy-efficient, and secure communication in such networks is therefore a major challenge.

Routing is a critical component of IoT network performance, as inefficient or unstable routing directly affects packet delivery, latency, energy consumption, and overall network lifetime. To address these challenges, the Internet Engineering Task Force introduced the IPv6 Routing Protocol for Low-Power and Lossy Networks (RPL). RPL is designed to support large-scale deployments by organizing nodes into a Destination-Oriented Directed Acyclic Graph (DODAG) and using lightweight control messaging mechanisms.

Although RPL is efficient and scalable, it was not designed with strong security guarantees. The protocol assumes cooperative behavior among nodes and relies heavily on locally exchanged routing information. These assumptions expose RPL networks to a wide range of routing attacks, particularly insider attacks launched by compromised nodes that behave in a protocol-compliant manner. Such attacks can severely degrade network performance or even lead to complete network failure.

Traditional security mechanisms such as cryptographic authentication, trust-based systems, and rule-based intrusion detection have been shown to be insufficient in RPL-based IoT networks. These approaches either introduce excessive overhead, fail to detect insider attacks, or lack

adaptability in dynamic environments. Recent advances in artificial intelligence (AI) and machine learning (ML) provide an opportunity to overcome these limitations by enabling adaptive, data-driven intrusion detection. This project proposes an AI-based framework to intelligently detect and mitigate routing attacks in RPL-based IoT networks.

2. Problem Statement

RPL-based IoT networks are inherently vulnerable to routing attacks such as rank manipulation, version number attacks, blackhole and grayhole attacks, Sybil attacks, copycat attacks, and routing table exhaustion attacks. These attacks exploit the lightweight design of RPL and can be executed without violating protocol specifications, making them difficult to detect using traditional security mechanisms.

Attack Category	Attack Type	Description	Impact on Network
Rank-Based Attacks	Decreased Rank Attack	Malicious node advertises an artificially low rank to attract traffic	Routing loops, congestion, packet loss
Rank-Based Attacks	Increased Rank Attack	Node advertises exaggerated rank values	Topology instability, frequent parent changes
Control Plane Attacks	Version Number Attack	Unauthorized increment of DODAG version number	Excessive global repairs, energy depletion
Forwarding Attacks	Blackhole Attack	Node drops all data packets passing through it	Severe packet delivery ratio degradation

Forwarding Attacks	Grayhole Attack	Node selectively drops packets	Stealthy performance degradation
Identity Attacks	Sybil Attack	Single node assumes multiple fake identities	Load imbalance, routing inconsistency
Identity Attacks	Copycat Attack	Node impersonates an existing legitimate node	Trust exploitation, routing disruption
Resource Exhaustion	Routing Table Overload Attack	Flooding DAO messages to exhaust memory	Node failure, routing collapse

Table 1: Classification of RPL Routing Attacks

Existing defense solutions face several limitations. Cryptographic approaches protect message authenticity but cannot prevent malicious behavior by authenticated nodes. Rule-based intrusion detection systems rely on static thresholds and predefined attack signatures, leading to high false positives or missed detections in dynamic network conditions. Trust-based mechanisms struggle against adaptive and intermittent attackers and incur additional communication overhead.

Aspect	**Traditional Defenses**	**AI-Based Defenses**
Detection Method	Rule-based / Threshold-based	Data-driven learning
Insider Attack Detection	Limited	High

Adaptability	Static	Adaptive
False Positives	High in dynamic networks	Lower with training
Resource Overhead	Moderate to High	Tunable (model-dependent)
Zero-Day Attack Handling	Poor	Better generalization
Scalability	Limited	High (with edge/federated learning)

Table 2: Comparison of Traditional and AI-Based Defense Mechanisms

There is a clear need for an intelligent, adaptive, and resource-efficient intrusion detection mechanism capable of identifying protocol-compliant routing attacks while preserving the energy efficiency and scalability of IoT networks.

3. Objectives of the Project

The primary objective of this project is to design, implement, and evaluate an AI-based intrusion detection and mitigation framework for RPL-based IoT networks. The project aims to analyze the behavior and impact of major RPL routing attacks, identify discriminative routing and traffic features, and train intelligent models capable of distinguishing between normal and malicious behavior. Additionally, the project seeks to evaluate the effectiveness of AI-based detection in comparison to traditional methods, with a particular focus on detection accuracy, adaptability, and energy efficiency.

4. Scope of the Project

This project focuses on securing the network layer of IoT systems that employ the RPL routing protocol. The work is conducted in a simulation environment to allow controlled experimentation and detailed performance analysis. The scope includes modeling multiple routing attacks, collecting network behavior data, developing AI-based intrusion detection models, and evaluating their impact on network performance metrics.

The project does not aim to modify the core RPL protocol. Instead, it proposes a complementary security layer that can be integrated with existing RPL implementations, making the approach practical for real-world deployment.

5. Project Methodology

The proposed hybrid methodology introduces a fundamentally different way of securing RPL-based IoT networks by shifting the focus from traditional attack-type classification to **routing behavior understanding and intent inference**. Existing approaches typically attempt to identify specific attacks such as rank manipulation or blackhole attacks by matching observed behavior to predefined attack signatures. However, such methods are limited in dynamic IoT environments, where attackers can easily modify their behavior to evade detection while still achieving malicious goals.

To address this limitation, the proposed methodology is built on two complementary concepts: **routing stability modeling** and **attack intent inference**.

Instead of asking

"Which attack is occurring?",

the system asks

"How is routing behavior deviating from stability, and what is the attacker trying to achieve?".

This shift allows the detection mechanism to remain effective even when attackers alter their techniques or combine multiple attack strategies.

The first component of the methodology is **routing stability modeling**. In a normal RPL network, routing behavior follows predictable patterns over time. Rank values change gradually, parent selection remains relatively stable, control message exchanges are periodic rather than bursty, and packet forwarding ratios remain consistent. These properties collectively define the *stable operating state* of an RPL network.

The proposed system captures this behavior by constructing a **Routing Stability Vector (RSV)** composed of multiple temporal metrics, including rank variation magnitude, parent switching frequency, control message burstiness, and packet forwarding consistency. Rather than analyzing these metrics independently, they are treated as a joint representation of network health.

$$RSV_i(t) = [\Delta Rank_i(t),\ PS_i(t),\ CM_i(t),\ PDR_i(t)]$$

where $\Delta Ranki(t)$ denotes the change in rank, *PSi(t)* represents the parent switching rate, *CMi(t)* is the control message frequency, and *PDRi(t)* is the packet delivery ratio. Under normal conditions, these values remain within a bounded range, reflecting stable routing behavior.

A machine learning model is trained using RSV data collected during normal network operation. The model learns a function f(·) that captures the normal routing pattern, and any deviation is quantified using an anomaly score. For example, in reconstruction-based models, the anomaly score is computed as

$$A_i(t) = \|RSV_i(t) - R\hat{S}V_i(t)\|$$

A self-supervised learning approach is then used to learn the normal evolution of the Routing Stability Vector over time. The model is trained only on benign network behavior, without requiring labeled attack data. This is particularly important in IoT environments, where labeled attack datasets are scarce and

unrealistic to obtain in real deployments. Once trained, the model continuously monitors the network and identifies deviations from learned stability patterns. Significant deviations indicate abnormal routing behavior, which is treated as a potential intrusion, regardless of whether the attack type has been previously observed.

The second component of the methodology is **attack intent inference**. After detecting anomalous behavior, the system does not immediately attempt to map it to a specific attack. Instead, it analyzes how the anomaly affects routing behavior to infer the attacker's underlying intent.

For example, anomalies that reduce rank values and attract traffic toward a node indicate a *traffic attraction intent*, while frequent topology changes and control message bursts suggest a *topology destabilization intent*. Selective packet drops without major routing disruption indicate *silent disruption*, whereas excessive control signaling points toward *resource exhaustion*. By categorizing attacks based on intent rather than technique, the system becomes resilient to variations and combinations of attacks.

To further enhance the novelty and practical relevance of the methodology, the framework introduces a new metric called **Attack Containment Radius (ACR)**. This metric quantifies how far the impact of a detected attack propagates within the RPL DODAG. Instead of treating attacks as binary events (present or absent), the ACR measures the spatial spread of routing instability from the malicious node across multiple hops. This allows the system to assess not only *whether* an attack exists, but also *how damaging* it is to the overall network structure.

$$ACR = \max\{h \mid A_j(t) > \theta,\ hop(j, attacker) = h\}$$

where hop(j,attacker) denotes the hop distance from the malicious node. This mathematical formulation enables not only detection of abnormal routing behavior but also interpretation of attacker intent and assessment of network-wide impact in a structured and measurable manner.

A small ACR indicates that the attack impact is localized, whereas a larger ACR signifies that the attack has propagated across multiple hops, affecting a wider portion of the network.

The hybrid nature of the methodology lies in its integration of **self-supervised learning**, **behavioral intent analysis**, and **topology-aware impact assessment**. Together, these components form a detection framework that is adaptive, lightweight, and suitable for resource-constrained IoT environments. The approach does not require modification of the RPL protocol itself and can operate as a complementary security layer, making it practical for real-world deployment.

In summary, this hybrid methodology is novel because it reframes RPL security from attack identification to behavior understanding, enabling robust detection of both known and previously unseen routing attacks.

How Hop Count Is Calculated

In an RPL-based IoT network, the **hop count** represents the number of intermediate nodes a packet passes through to reach from one node to another. In this project, hop count is used to measure the distance between a malicious node and other nodes in the network for calculating the Attack Containment Radius (ACR).

RPL organizes the network in a tree-like structure called a **DODAG**, where each node maintains information about its parent node and its position relative to the root. Using this structure, the hop count between two nodes can be calculated by tracing the routing path through parent relationships. Starting from a given node, the number of parent-to-parent transitions required to reach the malicious node (or a common ancestor) is counted. Each transition represents one hop.

In practical terms, during simulation, the hop count is obtained directly from the RPL routing information maintained by each node. Most RPL implementations already store either the hop distance to the root or the parent information, which can be used to determine hop distance between nodes. Therefore, no additional computation is required; the hop count is derived by counting the number of links along the routing path. This hop value is then used in the calculation of ACR to determine how far the impact of an attack has spread in the network.

CLEAR PROCESS WORKFLOW

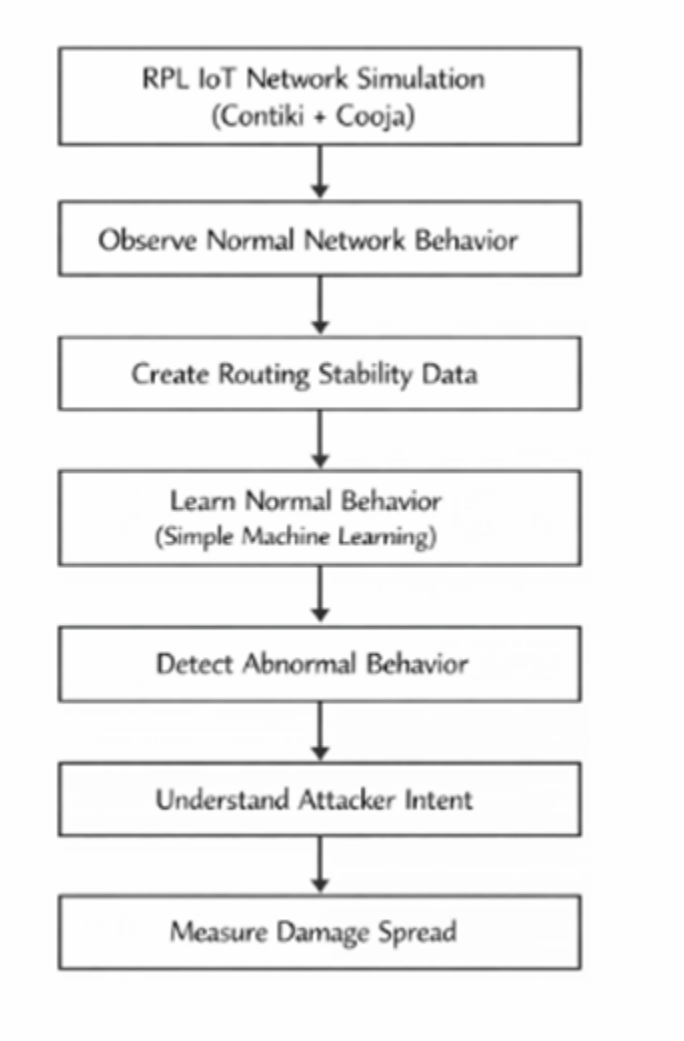

FIG. 1. Workflow of proposed methodology.

Step 1: Create the IoT Network

1. Use Cooja to create multiple IoT nodes

2. Enable RPL routing

3. One node acts as root

4. Network runs normally (no attacks)

Goal - Understand how a healthy network behaves.

Step 2: Observe Normal Behavior

1. We record simple things like:

2. How often routes change

3. How many control messages are sent

4. How many packets reach destination

5. These values form normal behavior data.

This will be our “baseline”.

Step 3: Create Routing Stability Data

We combine observations into a simple vector:

```cpp
Routing Stability Vector (RSV) =
[ rank change rate,
  parent change rate,
  control message count,
  packet delivery ratio ]
```

Goal - Represent network health using numbers.

Step 4: Learn Normal Behavior (ML Training)

1. Use Python + Scikit-learn

2. Train ML model only on normal data

Model learns what a healthy network behavior looks like.

Step 5: Introduce Attacks (Testing Phase)

Now we simulate attacks:

1. Rank manipulation

2. Packet dropping

3. Excessive control messages

Step 6: Detect Abnormal Behavior

ML model continuously checks RSV. If behavior is very different from normal → anomaly detected

Important - Instead of identifying attack names, we detect abnormal routing behavior

Step 7: Infer Attacker's Intent

Based on observed behavior:

Behavior Seen	Intent
Traffic pulled to node	Traffic attraction
Frequent route breaks	Network disruption
Silent packet loss	Stealth damage
Excessive messages	Resource exhaustion

Table 3: Intent Inference

This step will be rule-based.

Step 8: Measure How Big the Damage Is

We check how many nearby nodes are affected, how far the abnormal behavior spreads.

For this we will use Attack Containment Radius (ACR).

$$ACR = \max\{h \mid A_j(t) > \theta,\ hop(j, attacker) = h\}$$

Goal - Understand attack severity.

Step 9: Evaluate Performance

We compare:

1. Normal vs attack scenarios
2. With and without detection

Metrics:

1. Packet delivery ratio (PDR)
2. Delay
3. Energy usage
4. Detection accuracy

How the Proposed Methodology Differs from Existing Approaches

Most existing research in the domain of RPL security focuses on *attack-centric detection*, where the primary objective is to identify and classify specific routing attacks such as rank attacks, blackhole attacks, version number attacks, or Sybil attacks. These approaches typically rely on predefined rules, thresholds, or supervised machine learning models trained on labeled attack datasets. As a result, their effectiveness is strongly dependent on prior knowledge of attack patterns. When attackers modify their behavior, combine multiple attacks, or introduce new strategies, these systems often fail or produce high false positives.

In contrast, the proposed methodology adopts a *behavior-centric and stability-driven* perspective rather than an attack-centric one.

Instead of asking *"Which attack is occurring?",* the proposed approach asks *"Is the routing behavior stable or abnormal?".*

The system first learns the normal operational behavior of an RPL network by observing routing stability parameters such as rank variation, parent switching frequency, control message rate, and packet delivery ratio. These parameters are combined into a Routing Stability Vector, which captures the overall health of the routing topology. Any significant deviation from learned stability patterns is treated as suspicious, allowing the detection of abnormal behavior even when the attack type is unknown or previously unseen.

Another key difference lies in the learning strategy. Existing methods predominantly use **supervised learning**, which requires labeled datasets containing examples of all attack types. Such datasets are difficult to obtain in real IoT environments and limit practical applicability. The proposed methodology uses **a self-learning (normal-behavior-based) approach**, where the model is trained only on benign data. This makes the system more realistic, adaptive, and suitable for deployment in dynamic IoT networks where attack signatures cannot be predefined.

Furthermore, existing approaches usually stop at attack detection or classification, treating attacks as binary events. The proposed methodology goes a step further by introducing **attack intent inference**. Instead of identifying attacks by name, the system analyzes **how routing behavior is affected to infer the attacker's goal**, such as traffic attraction, topology disruption, silent packet dropping, or resource exhaustion. This intent-based abstraction makes the detection mechanism robust against variations and combinations of attacks and improves interpretability for network administrators.

Finally, most existing works evaluate attacks only through global performance metrics such as packet delivery ratio or delay. They do not quantify how far an attack spreads within the network. The proposed methodology introduces a novel metric called **Attack Containment Radius (ACR),** which measures the maximum hop distance from the malicious node to the farthest node exhibiting abnormal behavior.

This provides a clear, quantitative measure of attack severity and spatial impact, enabling localized mitigation rather than costly network-wide repairs.

Attack Resolution and Mitigation Strategy

Once abnormal routing behavior is detected and a node is identified as malicious, the proposed methodology focuses on lightweight and localized resolution, rather than aggressive network-wide actions. Since RPL-based IoT networks are resource-constrained, the goal of resolution is to limit the impact of the attack while preserving network stability and energy efficiency.

The primary resolution strategy involves logical isolation of the malicious node from routing decisions. After detection, the affected nodes are instructed to avoid selecting the malicious node as a preferred parent during RPL route formation. This is achieved by increasing the routing cost associated with the malicious node or marking it as unreliable, so that normal nodes naturally choose alternative, trusted parents. Importantly, this approach does not require modification of the RPL protocol; it operates as a complementary decision layer.

In addition to isolation, the Attack Containment Radius (ACR) is used to guide resolution decisions. If the ACR is small, indicating a localized attack, mitigation is applied only within the affected region of the network, avoiding unnecessary global repairs. If the ACR is large, stronger actions such as triggering local route recomputation or temporary suppression of routing updates from the malicious node can be applied. This adaptive resolution strategy ensures that the response is proportional to the severity of the attack.

This mitigation mechanism is implemented using C programming within the Contiki-NG operating system, which provides access to RPL routing metrics and parent selection logic. The Cooja simulator is used to observe routing changes and evaluate mitigation

effectiveness, while Python-based analysis tools are used to process detection results, apply ACR-based decisions, and evaluate post-mitigation network performance.

Overall, the resolution mechanism emphasizes containment rather than elimination, ensuring that data delivery continues through alternative paths while minimizing control overhead and energy consumption. By combining detection, intent inference, and hop-based impact analysis, the proposed system provides a practical and efficient approach to handling routing attacks in RPL-based IoT networks.

Comparison of Existing RPL Security Approaches and Proposed Methodology

Aspect	Existing Methodologies	Proposed Methodology
Detection Focus	Detect and classify specific attack types (e.g., rank attack, blackhole, Sybil)	Detect abnormal routing behavior irrespective of attack type
Core Question Addressed	“Which known attack is occurring?”	“Is routing behavior abnormal and what is the attacker trying to achieve?”
Detection Strategy	Attack-centric and signature-based	Behavior-centric and stability-based
Learning Approach	Mostly supervised learning requiring labeled attack data	Learning based on normal behavior (no attack labels required)

Dependence on Prior Knowledge	High – requires predefined attack patterns	Low – works even for unknown or modified attacks
Adaptability to New Attacks	Limited, requires retraining or new rules	High, detects deviations from normal behavior
Routing Parameters Used	Individual metrics analyzed independently	Multiple routing parameters combined into a Routing Stability Vector
Handling Insider Attacks	Often ineffective due to protocol-compliant behavior	Effective due to behavioral deviation detection
Attack Classification	Mandatory (attack name identification)	Not required; focuses on abnormality and intent
Intent Awareness	Not considered	Explicitly infers attacker intent (e.g., traffic attraction, disruption)
Impact Assessment	Indirect (PDR, delay, energy only)	Direct measurement using Attack Containment Radius (ACR)
Spatial Impact Analysis	Not addressed	Quantifies how far attack spreads in hops
False Positives in Dynamic Networks	Often high	Reduced due to stability-based learning
Protocol Modification	Sometimes required	Not required
Suitability for IoT Constraints	Moderate to low due to overhead	High, lightweight and practical
Practical Deployability	Limited due to reliance on attack signatures	High, realistic for real-world IoT environments

Table 4: Comparison between existing and proposed methodologies

6. Tools and Technologies

The proposed methodology will be implemented using a combination of simulation, programming, and data analysis tools suitable for IoT security research. The RPL-based IoT network will be simulated using the Contiki-NG operating system along with the Cooja network simulator, which provides a realistic environment for low-power and lossy networks without requiring physical hardware. The RPL routing protocol and attack behaviors will be implemented using the C programming language within the Contiki environment. Network logs and routing data generated during simulation will be exported and analyzed using Python, which will also be used to develop the anomaly detection models. Data processing and analysis will be carried out using standard Python libraries such as NumPy and Pandas, while machine learning models will be implemented using Scikit-learn. Network performance evaluation and result visualization will be performed using Matplotlib. The entire implementation will be carried out on a standard Linux-based system, making the approach practical and lightweight.

7. Performance Evaluation

The proposed framework will be evaluated using both security and network performance metrics. Intrusion detection effectiveness will be measured using accuracy, precision, recall, and F1-score. Network-level performance will be evaluated using packet delivery ratio, end-to-end delay, control overhead, energy consumption, and network lifetime. These metrics will provide insight into both the effectiveness and efficiency of the proposed approach.

Metric	Description	Significance
Packet Delivery Ratio (PDR)	Ratio of packets received to packets sent	Measures reliability
End-to-End Delay	Time taken for a packet to reach destination	Indicates latency

Control Overhead	Ratio of control packets to data packets	Measures routing efficiency
Energy Consumption	Energy used for communication and processing	Determines network lifetime
Network Lifetime	Time until first/critical node dies	Overall sustainability
Detection Accuracy	Correct attack classification rate	IDS effectiveness
Precision & Recall	Quality of detection decisions	False positive/negative analysis

Table 5: Performance Metrics Used for Evaluation

8. Project Timeline (January–May)

The project will be carried out over a period of five months, from January to May, following a structured and phased approach as outlined below.

Month	Project Phase	Detailed Activities
January	Literature Review & Problem Formulation	An in-depth review of existing research on RPL routing, IoT security challenges, and routing attacks will be conducted. The internal working of RPL, threat model, and system architecture will be finalized.

February	Simulation Setup & Attack Implementation	The RPL-based IoT network will be configured using Contiki-NG and Cooja. Various routing attacks will be implemented in a protocol-compliant manner.
March	Data Collection & AI Model Development	Network data will be collected from normal and attack scenarios. Feature extraction, preprocessing, and training of machine learning models will be performed.
April	Model Evaluation & Performance Analysis	The trained models will be evaluated on unseen data. Detection accuracy and network performance metrics will be analyzed and compared with traditional approaches.
May	Results Consolidation & Documentation	Results will be consolidated, conclusions drawn, and the final project report and presentation will be prepared.

Table 6: Project Timeline (January–May)

9. Expected Outcomes

The expected outcome of this project is the development of an effective and lightweight intrusion detection framework for RPL-based IoT networks that can identify abnormal routing behavior without relying on predefined attack signatures. The proposed approach is expected to accurately detect both known and previously unseen routing attacks by learning normal routing stability patterns. In addition, the intent-based analysis will help classify malicious behavior based on its impact on the network rather than specific attack names. The introduction of the Attack Containment Radius (ACR) is expected to provide a clear measure of how far an attack spreads within the network, enabling better understanding of attack severity. Overall, the system is expected to improve detection accuracy, reduce false positives, and enhance network reliability while maintaining low computational and energy overhead suitable for IoT environments.

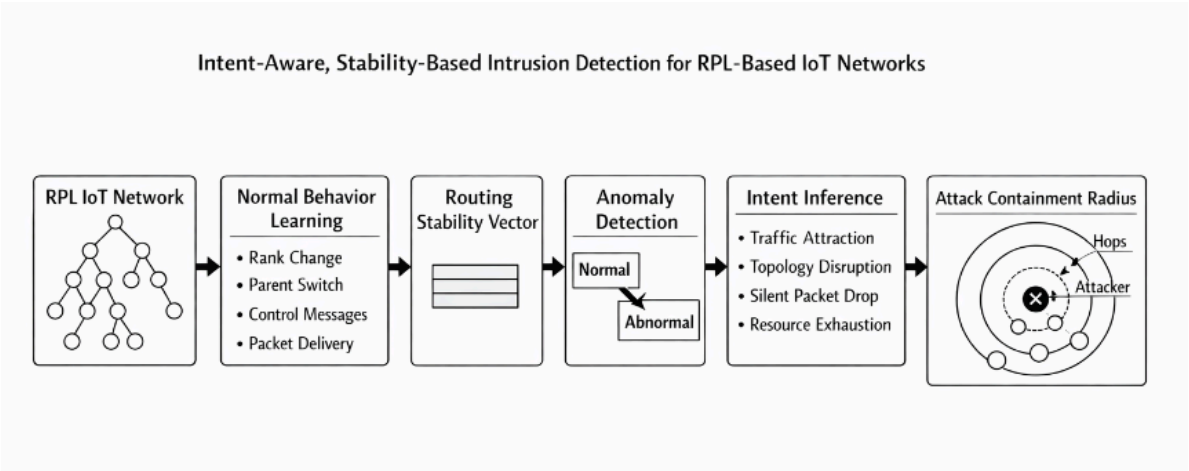

FIG. 2. Workflow Outcome of proposed methodology.

10. Conclusion

This project presents a novel and practical approach to securing RPL-based IoT networks by shifting the focus from attack-specific detection to behavior- and stability-based analysis. By learning normal routing behavior, detecting deviations, inferring attacker intent, and quantifying attack spread using hop-based analysis, the proposed methodology addresses key limitations of existing RPL security solutions. The approach does not require protocol modification or labeled attack data, making it realistic for deployment in dynamic and resource-constrained IoT networks. The proposed framework is both conceptually innovative and technically feasible for implementation, and it contributes toward building more resilient, adaptive, and trustworthy IoT routing systems.